

AutoRE: Bayesian-Optimization-based Automatic Reliability Enhancement Tool for Flow-based Microfluidic Biochips

Siyuan Liang¹, Yushen Zhang², Mengchu Li², Tsun-Ming Tseng², Ulf Schlichtmann², Tsung-Yi Ho¹

¹ The Chinese University of Hong Kong, ² Technical University of Munich

Abstract—As an emerging platform for biochemical experiments, flow-based microfluidic biochips are currently suffering from malfunctions caused by manufacturing defects, thereby having low yield. While many related studies have been conducted and reliability quantification models have been published, layout optimization methods are yet lacking. In this paper, we propose AutoRE, the first tool to automatically enhance reliability by optimizing layouts. AutoRE varies the layout within a certain range without changing its topology, and adopts Bayesian optimization (BO) to identify the most reliable variant. Experimental results demonstrate that AutoRE can efficiently and effectively improve the reliability across all testcases by around 40% on average.

I. INTRODUCTION

Recently, flow-based microfluidic biochips have emerged as an increasingly attractive platform for biochemical experiments [1]–[3]. Such coin-sized chips incorporate many miniaturized devices to carry out complex experiments. Compared to traditional cumbersome laboratory instruments, these chips enjoy many advantages including low reagents consumption, high automation degree, and high-sensitivity [3]. Such a chip consists of two layers: a flow layer containing flow channels and a control layer containing control channels. Reagents are transported in the flow channels, and the pressure in the control channels controls the transportation [4].

Due to the limitations of material and manufacturing processes, manufacturing defects can happen to these chips and cause malfunctions, resulting in low yield [5]. As the demand for more complex experiments increases, the integration level of these chips becomes higher, introducing significantly denser and longer channels. Consequently, this increases the risk of manufacturing defects, making the reliability of these chips an even more severe concern [6]–[8].

To improve the reliability of these chips, many related works have been conducted. Apart from some works aiming to achieve breakthroughs in material reliability and manufacturing processes, most previous works focus on enhancing the layouts of these chips. Zhu et al. proposed to construct a logic forest, enabling certain valves and channel segments to share the same functionalities, realizing fault tolerance [9]. Huang et al. proposed to construct backup channels in original layouts, providing each device-device connection with multiple possible paths [10]. Moradi et al. proposed to expand the routing fabric, guaranteeing the existence of multiple paths through different transposers [11]. Liang et al. proposed a more efficient coding strategy, and proposed to merge certain

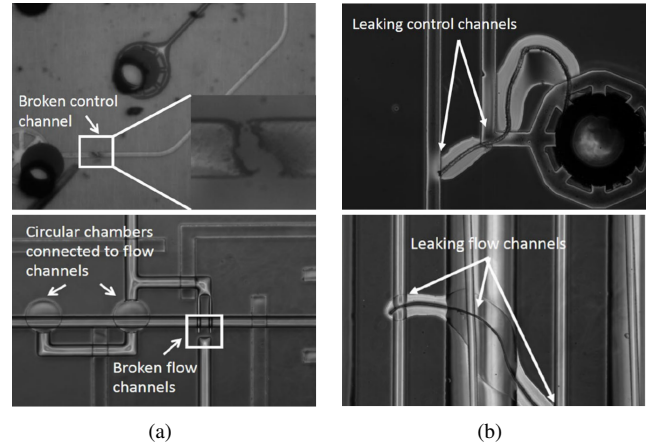


Fig. 1: Photos of common manufacturing defects [5]. (a) Blockage. (b) Leakage.

channel segments to reduce the density and length of channels in multiplexers [12].

These studies mainly focus on modifying the layouts to improve the reliability. Unfortunately, all of them lack evaluation methods of the reliability, making it impossible to quantify the degree of the reliability improvement. To tackle this problem, Liang et al. proposed reliability quantification models of two main types of manufacturing defects [13]. However, there is still a lack of layout optimization methods that utilize quantification models to optimize layouts for higher reliability.

In this paper, we propose AutoRE, the first automatic reliability enhancement tool for flow-based microfluidic biochips. Specifically, we first segment the original layout, and alter the segments within a certain range while maintaining the same topology. This way, we identify a feasible domain, in which we can explore among all feasible layout variants for higher reliability. We adopt Bayesian optimization (BO) to improve the efficiency of the exploration, and utilize reliability quantification models proposed in [13] to evaluate the reliability during the process. According to experimental results, AutoRE is an efficient tool that can effectively improve the reliability across all testcases by around 40% on average.

The rest of the paper is organized as follows: Section II introduces the background knowledge. Section III presents the overall flow of the proposed method. Section IV illustrates the identification of the feasible domain. Section V demonstrates the Bayesian-optimization-based reliability enhancement. Sec-

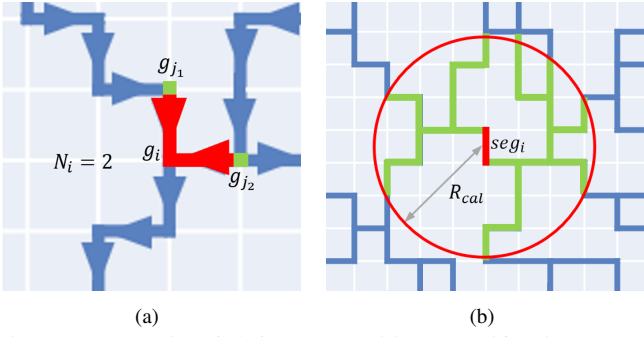


Fig. 2: Schematic of defect probability quantification models [13]. (a) Blockage. (b) Leakage.

tion VI shows the experimental results, and Section VII draws the conclusion.

II. BACKGROUND

A. Reliability Quantification Models

There are two main types of manufacturing defects: blockage and leakage [5]. The corresponding quantification models of their probabilities were proposed in [13].

1) Anti-Blockage Reliability

Blockage refers to the accidentally broken channel segments. Examples of broken control and flow channels are shown in Fig. 1(a). For control channels, blockage prevents pressure transportation, leading to losing control of valves; for flow channels, blockage prevents reagents from reaching reactors. Blockage is usually caused by environmental particles and imperfect silicon wafer molds.

According to previous studies [5], [6], blockage occurs in different parts of the layout almost randomly. Based on this, the blockage probability model proposed in [13] assumes every unit channel segment in the grid-based layout to have the same probability of being blocked. When a segment fails, the probability that its backup segments can take over the fluid transportation is the most important factor in determining the anti-blockage reliability. Hence, this model uses a propagation-based strategy to calculate $P_{blockage}$, the probability of layout failure due to blockage.

As shown in Fig. 2(a), the model first derives the directions of fluid transportation using the equivalent circuit model [14]. Following the derived directions, the model progressively calculates the probability of successful fluid transportation to different parts of the layout until it reaches all the ending points. For each grid point g_i , there are N_i adjacent upstream grid points g_j , providing N_i possible paths for fluid transportation. The probability of successful fluid transportation to g_i is determined by considering the likelihood that at least one of the N_i paths successfully transports fluid from g_j to g_i . The layout is considered well-functioning only if the fluid successfully reaches all the ending points. Therefore, $P_{blockage}$ is ultimately derived from the probability of fluid failing to reach all the ending points, and the anti-blockage reliability is then determined by $1 - P_{blockage}$.

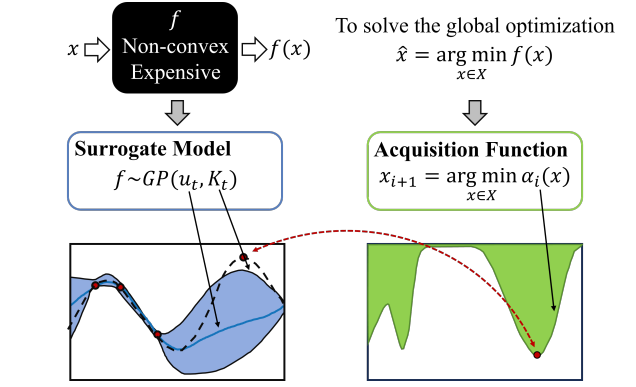


Fig. 3: Schematic of Bayesian optimization.

2) Anti-Leakage Reliability

Leakage refers to the unexpected connections between channels. Examples of leaking control and flow channels are shown in Fig. 1(b). For flow channels, leakage can make reagents unexpectedly infiltrate into other channels, resulting in catastrophic cross-contamination; for control channels, leakage transports pressure from one channel to the other, leading to unanticipated closure of valves. Leakage is usually caused by fiber pollutant and cracks in the layer.

According to previous studies [5], [6], the probability of leakage increases as the length of parallel channels increases or the distance between neighboring channels decreases. Based on this, the leakage probability model proposed in [13] assumes that each unit channel segment in the grid-based layout has the same probability to leak to another unit channel segment at a unit distance. Additionally, it assumes that leakage forms incrementally, unit distance by unit distance. In other words, the probability of leakage occurring between two unit channel segments decreases exponentially as the distance between them increases. This model combines leaking probabilities of each unit channel segment to derive $P_{leakage}$, the probability of layout failure due to leakage.

As shown in Fig. 2(b), for each unit channel segment seg_i , the model scans its nearby region within the range R_{cal} , and identifies all the nearby unit channel segments. The leaking probability of seg_i can then be calculated based on the probability of it leaking to any nearby unit channel segments. Since leakage may grow when high pressure is injected into the channels [5], the layout is considered well-functioning only if there is no leakage at all. Therefore, $P_{leakage}$ is ultimately derived from the probability of leakage occurring anywhere in the layout, and the anti-leakage reliability is then determined by $1 - P_{leakage}$.

B. Bayesian Optimization

As an efficient technique for black-box optimization method, Bayesian optimization (BO) seeks to quickly identify an input $\hat{x}$ that minimizes a function $f(x)$ within a domain $\mathbf{X}$, which is defined by finite lower and upper bounds for each variable.

$$\hat{x} = \underset{x \in \mathbf{X}}{\operatorname{argmin}} f(x) \quad (1)$$

The objective function $f(x)$ often lacks an analytical expression or derivative, limiting its evaluation to sampling at specific points x and obtaining potentially noisy responses. The feasible domain $\mathbf{X}$ is typically either a hyper-rectangle $\{x_i \in \mathbb{R}^d : a_i \leq x_i \leq b_i\}$ or a d -dimensional simplex $\{x_i \in \mathbb{R}^d : \sum_i x_i = 1\}$.

As shown in Fig. 3, BO relies on two critical components: (i) the surrogate model, which approximates the objective function; (ii) the acquisition function, which directs sampling to areas where an improvement over the current best observation f' is promising. The surrogate model is typically a Gaussian process (GP) [15], and many alternatives have also been proposed for better performance, such as random forest (RF), Bayesian additive regression trees (BART), and Bayesian multivariate adaptive regression splines (BMARS) [16]–[18]. A popular acquisition function is expected improvement (EI) [19], defined as the expected value of the utility function $u(x) = \max(0, f' - f(x))$ under the GP predictive distribution for f . Other acquisition functions like log expected improvement (logEI), probability of improvement (PI), and lower confidence bound (LCB) are also widely used in different works [19]–[21]. BO iterates between fitting a model and gathering additional data. Initially, the surrogate model $p(y|x, \mathcal{D}_t)$ is trained on a small set $\mathcal{D}_t$ of t samples from the domain $\mathbf{X}$. Subsequently, new points $\{x_{t+1}, x_{t+2}, \dots, x_T\}$ within the domain $\mathbf{X}$ are sequentially selected by optimizing an acquisition function S , which uses the current model $p(y|x, \mathcal{D}_t)$ as a cost-effective surrogate for the expensive objective f . This guided point selection, driven by the acquisition function, enables BO to consistently identify superior points, significantly enhancing efficiency compared to random selection. Each observed point x_{i+1} is evaluated by the function f , producing an observed result $f(x_{i+1})$. The result $(x_{i+1}, f(x_{i+1}))$ is added to the historical set $\mathcal{D}_i$, forming the new set $\mathcal{D}_{i+1}$, which updates the surrogate model $p(y|x, \mathcal{D}_{i+1})$ for the next observation point. Ultimately, the point $\hat{x}$ can be selected from $\mathcal{D}_T$, ensuring that the objective function f approaches or reaches the global optimum at $\hat{x}$.

III. OVERALL FLOW OF THE PROPOSED METHOD

The overall flow of AutoRE is shown in Fig. 4. The flow consists of two main stages: feasible domain identification, and BO-based reliability enhancement. We vary the given layout within the feasible domain to explore more reliable layout variants. In each round, we will vary the layout in only one of the x-axis and y-axis directions. The optimized layout from each round will serve as the initial solution for the next round, with the direction of variation being switched. This way, the relative locations of different parts will keep unchanged in each round, so that we can maintain the topology of the layout, preserving the functionality of the layout. Additionally, this approach simplifies the problem by reducing it from exploring a 2-D space to exploring multiple 1-D spaces, making the process more manageable and efficient.

Specifically, for a given layout, we first segment it into horizontal and vertical channel segments. In each round, only

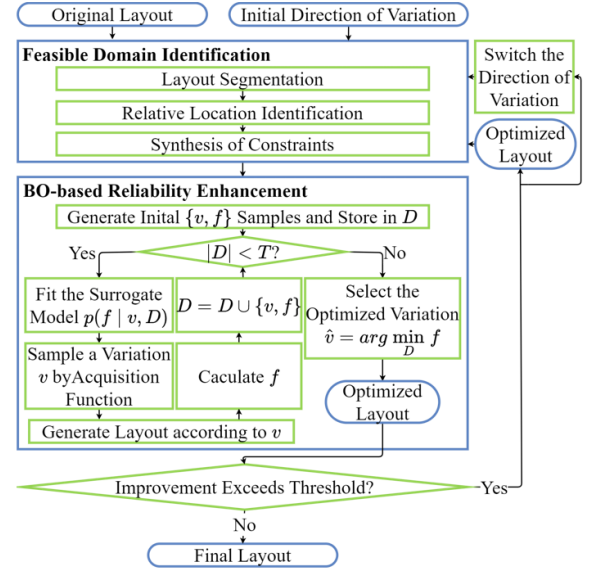


Fig. 4: The proposed overall flow of AutoRE.

the locations of the segments perpendicular to the current variation direction will be actively altered to shift along this direction, while the other segments will be extended or shortened to maintain the topology based on their connection relationships. Next, we develop an algorithm to identify the relative locations of the segments perpendicular to the current variation direction, and synthesize constraints accordingly, forming the feasible domain for the next stage. For example, when the variation direction is along the y-axis, only the y-axis locations of the horizontal channel segments will be actively altered during this round. For each horizontal channel segment, we will identify its nearest upper and lower neighbors, and only allow its y-axis location to be altered between these neighbors. This ensures that the relative locations of segments remain consistent during the variation.

Within the feasible domain, there can be thousands of possible layout variants, making it impractical to traverse all of them. Hence, we utilize BO to efficiently explore more reliable layout variants with only a limited number of attempts. During the exploration, we repeatedly sample layout variants and derive their corresponding costs. The distribution of the cost function will be predicted based on the obtained samples, and layout variants that are predicted to have lower costs will be more likely to be sampled in the future. This way, we can use much fewer attempts to find the more reliable layout variant, significantly improving the efficiency of exploration.

IV. FEASIBLE DOMAIN IDENTIFICATION

Given a layout, we need to first identify the feasible domain, in which we can vary the layout and explore the more reliable layout variant. For example, in Fig. 5(b), the green regions represent the feasible domains of horizontal channel segments $Hseg_1$ and $Hseg_4$, respectively.

A. Layout Segmentation

To make the variation efficient, we choose to segment the layout into horizontal and vertical channel segments, and alter these segments to derive the layout variants.

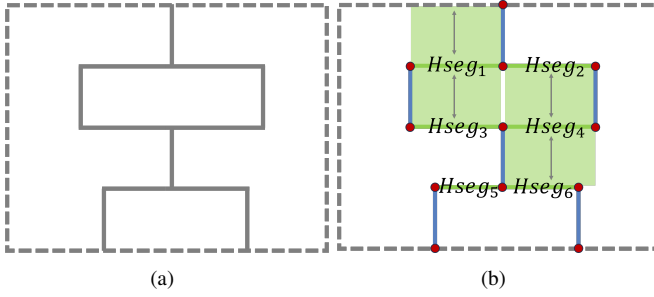


Fig. 5: Schematic of feasible domain identification. (a) The original layout. (b) Schematic of the identified feasible domains of two example horizontal channel segments.

Initially, we need to identify the terminals of segments. There are four types of points that can be terminals of segments: (i) starting points, where fluid is input into the design; (ii) ending points, where fluid is output from the design; (iii) branching points, which refer to points having more than two connections; (iv) corners, which refer to points with two connections forming a 90-degree angle. As shown in Fig. 5(b), the red dots represent the identified key points in the example layout shown in Fig. 5(a).

We then traverse all the terminals, identifying the horizontal and vertical connections among them as horizontal and vertical channel segments, respectively. As shown in Fig. 5(b), the green and blue segments between the red dots represent the identified horizontal and vertical channel segments, respectively. The connection relationships among these segments will also be recorded. Once segments in a certain direction are actively altered, segments in the other direction will be updated according to the connection relationships.

B. Relative Location Identification

During each round of variation, only the locations of segments perpendicular to the variation direction will be actively altered. To maintain the topology of the layout, we need to identify the relative locations of these segments, and only allow each segment to vary between its neighbors.

For example, when the variation direction is along the y-axis, for each horizontal channel segment $Hseg_i$, we will first search for all the horizontal channel segments overlapping with it in x-axis locations, and store them in $Overlap_i$. Whether or not two horizontal channel segments overlap in x-axis locations is determined by:

$$\mathbb{1}(\text{if overlap}) = \mathbb{1}(x_{i_{min}} \leq x_{j_{max}}) \mathbb{1}(x_{i_{max}} \geq x_{j_{min}}) \quad (2)$$

where $\mathbb{1}(\cdot)$ is the indicator function, $x_{i_{min}}$, $x_{i_{max}}$, $x_{j_{min}}$, and $x_{j_{max}}$ represent the x-axis locations of the left and right terminals of $Hseg_i$ and $Hseg_j$, respectively.

For each $Hseg_j$ in $Overlap_i$, we then calculate d_{ij} , the y-axis distance from $Hseg_i$ to $Hseg_j$:

$$d_{ij} = y_j - y_i \quad (3)$$

where y_i and y_j represent the y-axis locations of $Hseg_i$ and $Hseg_j$, respectively. $Hseg_j$ that correspond to the smallest positive d_{ij} and the largest negative d_{ij} are identified as $Hseg_i$'s nearest upper and lower neighbors, respectively. As

shown in Fig. 5(b), the upper and lower neighbors of $Hseg_4$ are $Hseg_2$ and $Hseg_6$, respectively. If there are no positive or negative d_{ij} , the upper or lower boundary of the layout will be regarded as the corresponding neighbor of $Hseg_i$. For example, in Fig. 5(b), the upper neighbor of $Hseg_1$ is the upper boundary. Besides, some $Hseg_j$ may be connected to the terminals of $Hseg_i$, making the corresponding $d_{ij} = 0$. Such a $Hseg_j$ is required to always keep the same y-axis location as $Hseg_i$, and it will be recorded in $Equal_i$ for later usage. Naturally, there can be at most two elements in $Equal_i$.

C. Synthesis of Constraints

We finally synthesize the constraints to identify the feasible domain for variation.

There are two types of constraints: (i) the upper and lower bounds of each segment; (ii) some segments need to have the same location in the variation direction. Taking the variation direction being along the y-axis as an example, we first synthesize type (i) constraints simply based on the neighbors of each $Hseg_i$. For type (ii) constraints, we refer to $Equal_i$. $Equal_i$ only contains $Hseg_j$ connected to $Hseg_i$, but the segments $Hseg_k$ connected to $Hseg_j$ are also required to have the same location as $Hseg_i$. Such relationship exists like a tree: $Hseg_i$ is the root, $Hseg_j$ in $Equal_i$ are the son nodes of $Hseg_i$, and $Hseg_k$ in $Equal_j$ are son nodes of $Hseg_j$. In this case, we recursively traverse nodes $Hseg_x$ and their corresponding $Equal_x$, until no new son nodes can be found. We then include all the upper and lower neighbors of segments in this tree into two sets UPs and $LOWs$, respectively. The type (i) constraint for $Hseg_i$ will be modified: the lowest segment in UPs and the upmost segment in $LOWs$ will become the new upper bound and lower bound, respectively. Type (i) constraints of other segments in this tree will be updated to let these segments have the same y-axis location as $Hseg_i$. These constraints build up the feasible domain for the variation, ensuring that layout variants within this domain maintain the same topology.

V. BAYESIAN-OPTIMIZATION-BASED RELIABILITY ENHANCEMENT

Since there can be thousands of possible layout variants within the previously identified feasible domain, it is impractical to traverse all of them. Hence, we utilize Bayesian optimization (BO) to efficiently explore more reliable layout variants.

As shown in Fig. 4, a set D is utilized to record all variations v and their corresponding costs f explored during the exploration. For a given feasible domain, we first randomly generate a few v and their corresponding layouts. These v and f will be recorded in D as the initial samples. When the size of D is smaller than the manually-set limit T , we will fit the surrogate model, sample a new v , derive the cost according to the corresponding layout, and update D . Once the size of D exceeds T , we will find the optimized v and its corresponding layout with the smallest cost. If this cost exceeds the improvement threshold, we will switch the direction of variation, pass the optimized layout to the

next round, and restart from the feasible domain identification stage. Otherwise, this optimized layout will be output as the final layout.

A. Problem Model

In a variation $v : \{v[1], v[2], \dots, v[n]\}$, each element $v[i]$ relates to the location of a to-be-actively-altered segment, and n represents the number of such segments. Here, we take the variation direction being along the y-axis as an example without loss of generality. As the identified feasible domain only contains the relative locations of segments, the range of the possible location of each segment is related to other segments, thereby being unfixed in value. This makes it impractical to directly use the exact location of each segment in the variation v . To tackle this problem, for each to-be-actively-altered segment $Hseg_i$, we use a ratio between its upper and lower bounds as its corresponding $v[i]$. This way, each $v[i] \in [0, 1]$, enabling the exploration.

With a sampled v , we construct linear equations to solve the specific locations $y : \{y[1], y[2], \dots, y[n]\}$, thereby deriving the corresponding layout variant. As a minimum distance d_{min} should be kept among channels, if both the upper and lower bounds are other segments $Hseg_{i_{up}}$ and $Hseg_{i_{low}}$, the real upper bound and lower bound of $y[i]$ should be $Realup_i = (y[i_{up}] - d_{min})$ and $Reallow_i = (y[i_{low}] + d_{min})$, respectively. For each linear equation, there are four possible scenarios, here we use b_{max} to represent the maximum boundary of the design in the current variation direction: (i) if both the upper and lower bounds are other segments:

$$y[i] - Reallow_i = v[i](Realup_i - Reallow_i) \quad (4)$$

(ii) if only the upper bound is another segment:

$$y[i] = v[i]Realup_i \quad (5)$$

(iii) if only the lower bound is another segment:

$$y[i] - Reallow_i = v[i](b_{max} - Reallow_i) \quad (6)$$

(iv) if neither of the bounds is another segment:

$$y[i] = v[i]b_{max} \quad (7)$$

We use the reliability evaluated by the quantification models proposed in [13] of the corresponding layout as the objective function f for v , which is defined as:

$$f(v) = (1 - w)P_{blockage} + wP_{leakage} \quad (8)$$

where $w \in [0, 1]$ is a user-defined parameter used to combine the anti-blockage and anti-leakage reliabilities. Our goal is to efficiently find the optimized variation $\hat{v}$ within only a limited number of attempts:

$$\hat{v} = \underset{v_i \in D}{\operatorname{argmin}} f(v_i), |D| \leq T \quad (9)$$

Specifically, we employ a BO method named SMAC3 to address the problem [22].

B. Surrogate Model

Unlike traditional BO methods [7], [23], we use random forest (RF) [16] as the surrogate model instead of Gaussian processes (GP). RF consists of collections of regression trees, which are similar to decision trees but have real values at their leaves rather than class labels [24]. As a non-parametric

approach, RF performs better on highly non-linear objective functions and is easy to parallelize [25], [26].

We construct an RF as a set of B regression trees, each built on m data points $\{v, f(v)\}$ sampled with replacement from the entire feasible domain. At each split point of each tree, a random subset of $[d \cdot p]$ of the d algorithm parameters is considered for splitting, where p is a parameter set as $p = \frac{5}{6}$. Another parameter is m_{min} , the minimum number of data points required in a node for it to be split further, which we set as $m_{min} = 10$. Finally, the number of trees is set as $B = 10$ to keep computational overhead low [27]. The RF's predictive mean μ_v and variance σ_v^2 for a new configuration v are computed as the empirical mean and variance of the individual trees' predictions for v .

C. Acquisition Function and Optimizer

Traditional expected improvement (EI) can suffer from vanishing gradients in high-dimensional spaces, making it difficult to optimize [20]. Therefore, we choose log expected improvement (logEI) as the acquisition function, which is defined as [20]:

$$\log EI_{f(v)_{best}}(v) = \log_h((\mu(v) - f(v)_{best})/\sigma(x)) + \log(\sigma(x)) \quad (10)$$

where $\log_h$ is mathematically equivalent to $\log \circ h$, and $f(v)_{best}$ is the best cost observed so far. This acquisition function ensures more stable gradients, leading to better results [28]. Meanwhile, we use a heuristic local search strategy to further improve the efficiency.

VI. EXPERIMENTAL RESULTS

We demonstrate the performance of AutoRE from the perspectives of effectiveness and efficiency, respectively. All the experiments are carried out on an Intel(R) Xeon(R) Gold 6226R 2.90GHz Linux machine with 256 GB RAM.

A total of six testcases are used in our experiments: Syn1, Syn2 and Syn3 are synthesized based on the design in [29]; Real1, Real2 and Real3 are real chip designs from [10]–[12]. The reliability quantification models from [13] are utilized to evaluate the anti-blockage and anti-leakage reliabilities. According to our manufacturing experience, in the quantification models, pb_{unit} , the probability of a unit channel segment being blocked, is set as 2%; pl_{unit} , the probability of each unit channel segment leaking to any other unit channel segment at a unit distance, is set as 0.4%. For the settings of AutoRE, we let the initial variation direction to be along the y-axis, allow 150 explorations in each round of variation, and set the improvement threshold as 1%. For each testcase, we conduct three different experiments with w set as 0, 0.5, and 1, respectively.

A. Experiments on the Effectiveness

To start with, we test the effectiveness of AutoRE in enhancing the reliability of different layouts. The results are shown in TABLE I. It can be observed that AutoRE significantly enhances both the anti-blockage and anti-leakage reliabilities. Specifically, the anti-blockage reliability is improved by 42.53%, and the anti-leakage reliability is improved by 37.59% on average across all testcases.

TABLE I: Performance of Reliability Enhancement for Different Testcases

Testcase	Size	Channel Density	Improvement of Anti-Blockage Reliability				Improvement of Anti-Leakage Reliability				Average Runtime (s)
			$w = 0$	$w = 0.5$	$w = 1$	Average	$w = 0$	$w = 0.5$	$w = 1$	Average	
Syn1	15×11	20.00%	1.22%	1.22%	-0.35%	0.69%	4.07%	4.07%	6.64%	4.93%	48.47
Syn2	30×20	17.50%	152.15%	94.08%	33.44%	93.22%	-14.97%	25.31%	27.23%	12.52%	38.45
Syn3	65×40	13.10%	45.53%	37.63%	22.72%	35.29%	65.63%	188.74%	194.07%	149.48%	1425.57
Real1 [12]	39×31	14.89%	52.85%	52.85%	52.85%	52.85%	9.85%	9.85%	9.85%	9.85%	24.14
Real2 [10]	20×20	11.50%	19.99%	19.99%	-11.21%	9.59%	-0.02%	-0.02%	2.37%	0.78%	122.16
Real3 [11]	60×49	14.46%	126.09%	134.38%	-69.91%	63.52%	-23.80%	-1.43%	169.10%	47.96%	947.33
Average	1319	15.24%	66.31%	56.69%	4.59%	42.53%	6.79%	37.75%	68.21%	37.59%	434.40

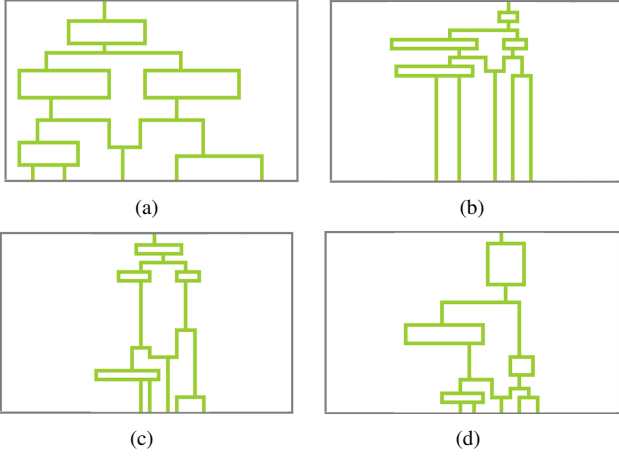


Fig. 6: Layouts of testcase Syn3. (a) The original layout. (b) The optimized layout with $w = 0$. (c) The optimized layout with $w = 0.5$. (d) The optimized layout with $w = 1$.

Additionally, the choice of w greatly influences the results. Generally, the closer w is to 0, the more AutoRE focuses on improving the anti-blockage reliability. This demonstrates AutoRE's flexibility in adapting to users' varying emphases on the two types of reliabilities. Conversely, as w increases, AutoRE tends more to enhance the anti-leakage reliability. Layout variants of Syn3 derived with different w settings are shown in Fig. 6. In extreme scenarios, such as Real3 with $w = 0$ and $w = 1$, AutoRE may even sacrifice one type of the reliability to achieve greater improvement in the other.

B. Experiments on the Efficiency

We also test the efficiency of AutoRE. According to the average runtime shown in TABLE I, AutoRE can complete the reliability enhancement for all testcases within 1500 s, which is a satisfactory speed for practical use.

To better demonstrate the efficiency of the BO-based exploration utilized by AutoRE, we compare it with the random exploration. Fig. 7 shows the result of the comparison using testcase Syn3. In the figures, each dot represents an exploration, with the y-axis value indicating the ratio between the current cost and the original cost, and the lines are drawn using the average of every ten explorations. It is evident that no matter what value w is set to adapt to users' different emphases on the two types of reliabilities, the BO-based exploration always consistently reduces the cost of

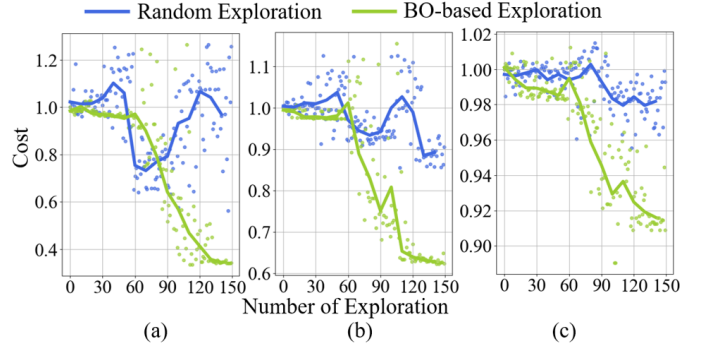


Fig. 7: Comparison of the efficiency between random exploration and the proposed BO-based exploration on testcase Syn3. (a) $w = 0$. (b) $w = 0.5$. (c) $w = 1$.

subsequent explorations, whereas the random exploration does not. Additionally, the lowest cost obtained through the BO-based approach are significantly better than those achieved by the random exploration. This demonstrates that the BO-based exploration is a more purposeful and efficient method for exploring within the feasible domain compared to the random exploration, proving AutoRE to be a highly efficient approach.

VII. CONCLUSION

In this paper, we have proposed the first automatic reliability enhancement tool for flow-based microfluidic biochips, named AutoRE. We have segmented the original layout and allowed segments to vary within a specified range while preserving the same topology. This approach has helped us identify a feasible domain, enabling us to explore among all feasible layout variants to achieve higher reliability. To improve the efficiency of our exploration, we have utilized Bayesian optimization (BO) and apply the reliability quantification models proposed [13] to evaluate the reliability of layout variants throughout the process. Experimental results have demonstrated that AutoRE is an efficient tool, effectively enhancing the reliability across all testcases by around 40% on average.

VIII. ACKNOWLEDGMENT

The research work described in this paper was conducted in the JC STEM Lab of Intelligent Design Automation funded by The Hong Kong Jockey Club Charities Trust. This work is jointly supported by the Research Grants Council of Hong Kong SAR (No. CUHK14217022), and the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation, No. 515003344).

REFERENCES

- [1] S. Liang, M. Li, T.-M. Tseng, U. Schlichtmann, and T.-Y. Ho, "Combinatorial-coding-based high-performance microfluidic control multiplexer: Design, synthesis, and adaptation," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2024.
- [2] M. Li, H. Gu, Y. Zhang, S. Liang, H. Gasvoda, R. Altay, I. E. Araci, T.-M. Tseng, T.-Y. Ho, and U. Schlichtmann, "Late breaking results: Efficient built-in self-test for microfluidic large-scale integration (mlsi)," in *ACM/IEEE Design Automation Conference (DAC)*, 2024.
- [3] K. Hu, K. Chakrabarty, and T.-Y. Ho, *Computer-aided design of microfluidic very large scale integration (mVLSI) biochips*. Springer, 2017.
- [4] S. Liang, Y. Zhang, R. Altay, H. Gasvoda, M. Li, I. E. Araci, T.-M. Tseng, U. Schlichtmann, and T.-Y. Ho, "Lamux: Optimized logic-gate-enabled high-performance microfluidic multiplexer design," 2024.
- [5] K. Hu, F. Yu, T.-Y. Ho, and K. Chakrabarty, "Testing of flow-based microfluidic biochips: Fault modeling, test generation, and experimental demonstration," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 33, no. 10, pp. 1463–1475, 2014.
- [6] K. Hu, T.-Y. Ho, and K. Chakrabarty, "Test generation and design-for-testability for flow-based mvlsi microfluidic biochips," in *2014 IEEE 32nd VLSI Test Symposium (VTS)*, pp. 1–6, IEEE, 2014.
- [7] S. Liang, R. Fu, M. Li, T.-M. Tseng, U. Schlichtmann, and T.-Y. Ho, "Raber: Reliability-aware bayesian-optimization-based control layer escape routing for flow-based microfluidics," in *IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024.
- [8] M. Li, Y. Zhang, J. Y. Lee, H. Gasvoda, I. E. Araci, T.-M. Tseng, and U. Schlichtmann, "Integrated test module design for microfluidic large-scale integration," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 6, pp. 1939–1950, 2022.
- [9] Y. Zhu, B. Li, T.-Y. Ho, Q. Wang, H. Yao, R. Wille, and U. Schlichtmann, "Multi-channel and fault-tolerant control multiplexing for flow-based microfluidic biochips," in *2018 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pp. 1–8, IEEE, 2018.
- [10] W.-L. Huang, A. Gupta, S. Roy, T.-Y. Ho, and P. Pop, "Fast architecture-level synthesis of fault-tolerant flow-based microfluidic biochips," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2017, pp. 1667–1672, IEEE, 2017.
- [11] Y. Moradi, M. Ibrahim, K. Chakrabarty, and U. Schlichtmann, "Fault-tolerant valve-based microfluidic routing fabric for droplet barcoding in single-cell analysis," in *2018 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1484–1487, IEEE, 2018.
- [12] S. Liang, M. Li, T.-M. Tseng, U. Schlichtmann, and T.-Y. Ho, "Comux: Combinatorial-coding-based high-performance microfluidic control multiplexer design," in *Proceedings of the 41st IEEE/ACM International Conference on Computer-Aided Design*, pp. 1–9, 2022.
- [13] S. Liang, M. Lian, M. Li, T.-M. Tseng, U. Schlichtmann, and T.-Y. Ho, "Armm: Adaptive reliability quantification model of microfluidic designs and its graph-transformer-based implementation," in *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pp. 1–9, IEEE, 2023.
- [14] W. Ji, T.-Y. Ho, J. Wang, and H. Yao, "Microfluidic design for concentration gradient generation using artificial neural network," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 10, pp. 2544–2557, 2019.
- [15] C. E. Rasmussen and C. K. I. Williams, *Gaussian Processes for Machine Learning*. The MIT Press, 11 2005.
- [16] F. Hutter, H. H. Hoos, and K. Leyton-Brown, "Sequential model-based optimization for general algorithm configuration," in *Learning and Intelligent Optimization: 5th International Conference, LION 5, Rome, Italy, January 17-21, 2011. Selected Papers 5*, pp. 507–523, Springer, 2011.
- [17] B. Lei, T. Q. Kirk, A. Bhattacharya, D. Pati, X. Qian, R. Arroyave, and B. K. Mallick, "Bayesian optimization with adaptive surrogate models for automated experimental design," *Npj Computational Materials*, vol. 7, no. 1, p. 194, 2021.
- [18] H. A. Chipman, E. I. George, and R. E. McCulloch, "Bart: Bayesian additive regression trees," 2010.
- [19] D. R. Jones, M. Schonlau, and W. J. Welch, "Efficient global optimization of expensive black-box functions," *Journal of Global optimization*, vol. 13, no. 4, pp. 455–492, 1998.
- [20] S. Ament, S. Daulton, D. Eriksson, M. Balandat, and E. Bakshy, "Unexpected improvements to expected improvement for bayesian optimization," *Advances in Neural Information Processing Systems*, vol. 36, pp. 20577–20612, 2023.
- [21] N. Srinivas, A. Krause, S. M. Kakade, and M. Seeger, "Gaussian process optimization in the bandit setting: No regret and experimental design," *arXiv preprint arXiv:0912.3995*, 2009.
- [22] M. Lindauer, K. Eggenberger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, and F. Hutter, "Smac3: A versatile bayesian optimization package for hyperparameter optimization," *Journal of Machine Learning Research*, vol. 23, no. 54, pp. 1–9, 2022.
- [23] R. Fu, J. Huang, M. Wang, Y. Nobuyuki, B. Yu, T.-Y. Ho, and O. Chen, "Bomig: A majority logic synthesis framework for aqfp logic," in *2023 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, pp. 1–2, IEEE, 2023.
- [24] L. Breiman, "Random forests," *Machine learning*, vol. 45, pp. 5–32, 2001.
- [25] J. Snoek, H. Larochelle, and R. P. Adams, "Practical bayesian optimization of machine learning algorithms," *Advances in neural information processing systems*, vol. 25, 2012.
- [26] R. Iranzad and X. Liu, "A review of random forest-based feature selection methods for data science education and applications," *International Journal of Data Science and Analytics*, pp. 1–15, 2024.
- [27] M. Birattari, Z. Yuan, P. Balaprakash, and T. Stützle, "F-race and iterated f-race: An overview," *Experimental methods for the analysis of optimization algorithms*, pp. 311–336, 2010.
- [28] F. Hutter, H. H. Hoos, K. Leyton-Brown, and K. Murphy, "Time-bounded sequential parameter optimization," in *Learning and Intelligent Optimization: 4th International Conference, LION 4, Venice, Italy, January 18-22, 2010. Selected Papers 4*, pp. 281–298, Springer, 2010.
- [29] K. Hu, T. A. Dinh, T.-Y. Ho, and K. Chakrabarty, "Control-layer optimization for flow-based mvlsi microfluidic biochips," in *Proceedings of the 2014 International Conference on Compilers, Architecture and Synthesis for Embedded Systems*, pp. 1–10, 2014.